МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу “Компьютерная графика”

Камера и освещение

Выполнил: Н. Д. Князьков
Группа: М8О-303БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: реализовать «камеру» с возможностью её перемещения и освещение по модели Блинна-Фонга с использованием нескольких типов источников света. Программа написана на C++ с использованием Vulkan, GLFW и ImGui на основе кода лабораторной работы №1.

Объект: правильный икосаэдр (вариант 11 из лабораторной работы №1).

Базовое задание: отрисовка объекта относительно камеры с единственным направленным источником освещения. Параметры камеры, источника, материала по модели Блинна-Фонга и рассеянный свет (ambient light) изменяются в интерфейсе ImGui. У вершин задан атрибут нормали, нормали генерируются процедурно, освещение считается во фрагментном шейдере.

Выполненные дополнительные задания (1–4):

1. Внешние данные разделены на два набора дескрипторов: набор сцены (uniform-буфер с параметрами сцены и storage-буферы с массивами источников) и набор модели (матрица трансформации и параметры материала).
2. Освещение массивом точечных источников из storage-буфера с затуханием по закону обратных квадратов; добавление, удаление и изменение источников в интерфейсе.
3. Освещение массивом прожекторов из storage-буфера с затуханием по закону обратных квадратов и мягкими краями (внутренний и внешний углы); добавление, удаление и изменение источников в интерфейсе.
4. Визуализация источников с возможностью показа и скрытия: яркий куб цвета источника для точечных источников и прожекторов, направление и угол прожектора нарисованы белыми линиями (конус). Фигуры имеют постоянный цвет и не освещаются по модели Блинна-Фонга.

Дополнительно на сцену добавлен пол, а три икосаэдра из лабораторной работы №1 движутся по траектории, поэтому освещение от разных источников меняется в реальном времени.

Метод решения

Нормали. Нормаль — единичный вектор, перпендикулярный поверхности. Для каждой грани икосаэдра с вершинами a, b, c (перечислены против часовой стрелки, если смотреть снаружи) нормаль считается векторным произведением рёбер:

$$n = normalize((b - a) \times (c - a))$$

Чтобы каждая из 20 граней освещалась как плоская, вершины между гранями не делятся: в вершинном буфере 60 вершин (позиция + нормаль). У пола нормаль (0, −1, 0), так как ось Y направлена вниз. В вершинном шейдере нормаль переводится в мировые координаты матрицей $(M^{-1})^T$ — обратной транспонированной к матрице модели, иначе при неравномерном растяжении она перестала бы быть перпендикулярной поверхности. Во фрагментном шейдере интерполированная нормаль снова нормируется.

Камера. Камера задаётся позицией c, углами поворота ψ (yaw, вокруг оси Y) и θ (pitch, вокруг оси X) и углом обзора. Её матрица модели C = T(c)·Ry(ψ)·Rx(θ), а матрица вида — обратная к ней, поэтому множители идут в обратном порядке и с противоположными параметрами:

$$V = C^{-1} = Rx(-\theta) \cdot Ry(-\psi) \cdot T(-c)$$

Для перемещения с клавиатуры (W/S — вперёд/назад, A/D — влево/вправо, Q/E — вниз/вверх, стрелки — поворот) направления берутся из поворота камеры: вперёд = (sin ψ·cos θ, −sin θ, cos ψ·cos θ), вправо = (cos ψ, 0, −sin ψ). Матрицы проекции взяты из лабораторной работы №1, в шейдер передаётся произведение view_projection = P·V, а матрица модели — отдельно.

Модель освещения Блинна-Фонга. Для каждого источника цвет точки поверхности:

$$C = I \cdot (\cos\theta \cdot Kd + Ks \cdot (\cos\varphi)^{\wedge}\alpha)$$

$$\cos\theta = n \cdot l, \quad h = (l + v) / |l + v|, \quad \cos\varphi = n \cdot h$$

Здесь n — нормаль, l — направление на источник, v — направление на камеру, I — цвет источника, умноженный на интенсивность, Kd — альбедо, Ks — цвет блика, α — коэффициент блеска (чем больше α, тем меньше и ярче блик). Если $n \cdot l \le 0$, источник находится за поверхностью и вклад равен нулю. Рассеянный свет добавляется как Ia·Ka (в качестве Ka берётся

альбедо), вклады всех источников складываются. Все вычисления выполняются во фрагментном шейдере в мировых координатах.

Типы источников. Направленный источник (солнце) задаётся направлением лучей d, для него l = −d и затухания нет. Точечный источник задаётся позицией p: l = (p − x)/D, D = |p − x|, а освещённость падает по закону обратных квадратов — множитель $1/D^2$. Прожектор — точечный источник с направлением d и двумя углами: внутри внутреннего угла θi свет полный, снаружи внешнего угла θo — отсутствует, между ними плавно убывает:

$$s = clamp((\cos\gamma - \cos\theta o) / (\cos\theta i - \cos\theta o), 0, 1), \quad \cos\gamma = (-l)\cdot d$$

В шейдер передаются косинусы углов, чтобы не вычислять их для каждого пикселя. Если углы равны, край пятна получается резким.

```
vec3 blinnPhong(vec3 n, vec3 v, vec3 l, vec3 light) {
    float cos_theta = dot(n, l);
    if (cos_theta <= 0.0) return vec3(0.0);
    vec3 h = normalize(l + v);
    float cos_phi = max(dot(n, h), 0.0);
    vec3 diffuse = model_uniforms.albedo * cos_theta;
    vec3 specular = model_uniforms.specular *
                    pow(cos_phi, model_uniforms.alpha);
    return light * (diffuse + specular);
}
```

Передача данных в шейдеры (доп. задание 1). Данные разделены по частоте изменения на два набора дескрипторов. Набор сцены привязывается один раз за кадр, набор модели — перед отрисовкой каждого объекта (vkCmdBindDescriptorSets с firstSet = 1, набор 0 при этом остаётся привязанным).

set / binding	**Тип**	**Содержимое**
0 / 0	uniform-буфер, std140	SceneUniforms: view_projection, позиция камеры, направленный источник, рассеянный свет, число точечных источников и прожекторов
0 / 1	storage-буфер, std430	PointLights: массив {position, color}, 32 байта на источник
0 / 2	storage-буфер, std430	SpotLights: массив {position, color, direction, cos θi, cos θo}, 64 байта на источник
1 / 0	uniform-буфер, std140	ModelUniform: матрица модели, альбедо, цвет блика, коэффициент блеска (96 байт)

По правилам std140 и std430 vec3 выравнивается на 16 байт, поэтому в структурах на C++ после каждого vec3 добавлены 4 байта выравнивания, а float или uint может занять место сразу за vec3 (например, point_light_count

лежит по смещению 124, сразу после ambient_light). Массивы источников лежат в storage-буферах (ключевое слово buffer в GLSL), поэтому размер массива в шейдере не задаётся. Если источников становится больше, чем помещается в буфер, буфер пересоздаётся в два раза больше и дескриптор в наборе сцены переписывается через vkUpdateDescriptorSets; это безопасно, потому что к моменту записи команд GPU уже закончил предыдущий кадр.

Визуализация источников (доп. задание 4). Фигуры источников рисуются отдельными конвейерами со своими шейдерами light.vert и light.frag, которые выводят постоянный цвет без освещения. Матрица модели и цвет фигуры (80 байт) передаются push-константами (vkCmdPushConstants) прямо в командный буфер перед каждой отрисовкой — так не нужно создавать буферы и наборы дескрипторов под меняющееся число источников. Точечные источники и прожекторы показываются кубом со стороной 0,3 цвета источника. Конус прожектора рисуется белыми линиями (топология LINE_LIST): ось, 8 образующих и окружности внешнего и внутреннего углов на расстоянии 1,5 от источника (радиус $1{,}5 \cdot \operatorname{tg} \theta$). Вершины линий каждый кадр считаются на CPU в мировых координатах и копируются в вершинный буфер. У этих конвейеров другой pipeline layout (с push-константами), поэтому набор сцены после их привязки привязывается заново.

Используемые объекты Vulkan:

- VkBuffer и VmaAllocation (Vulkan Memory Allocator) — вершинные и индексные буферы икосаэдра, пола и куба, uniform-буфер сцены, по uniform-буферу на объект, два storage-буфера источников, вершинный буфер линий конусов.
- VkDescriptorSetLayout — два описания наборов (сцена и модель); VkDescriptorPool на 5 наборов (5 uniform-буферов и 2 storage-буфера); VkDescriptorSet — один набор сцены и 4 набора объектов.
- VkPipelineLayout — для освещаемых объектов (наборы 0 и 1) и для визуализации источников (набор 0 и диапазон push-констант); VkPipeline — три конвейера: освещаемые объекты, кубы источников, линии конусов.
- VkShaderModule — шейдеры basic.vert/basic.frag (освещение) и light.vert/light.frag (постоянный цвет), скомпилированные в SPIR-V компилятором glslc.

– VkCommandBuffer — в render() копируются данные в буферы, затем записываются команды: render pass, привязка конвейера и набора сцены, для каждого объекта — привязка его вершин, индексов и набора модели и vkCmdDrawIndexed; затем, если включено, кубы и конусы источников.

Результаты

Программа работает в реальном времени, все параметры камеры, источников и материалов меняются в окне ImGui «Lab 2», камерой можно управлять с клавиатуры. Слои валидации Vulkan не выводят ошибок ни во время работы, ни при закрытии программы.

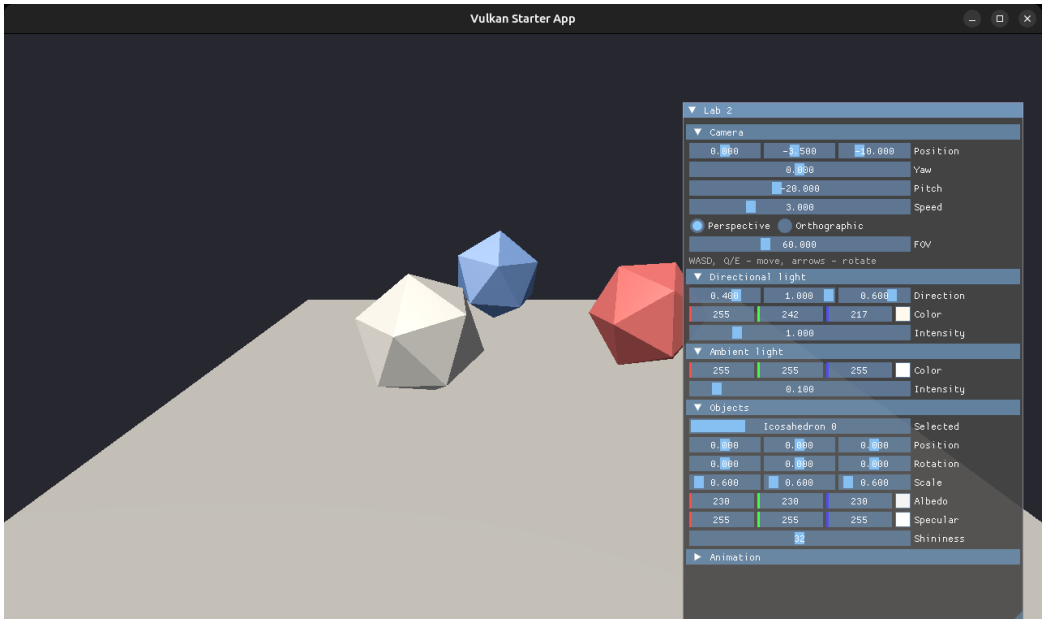


Рисунок 1 — базовое задание: три икосаэдра и пол освещены одним направленным источником и рассеянным светом; грани освещены по-разному, потому что у каждой грани своя нормаль. В окне интерфейса — параметры камеры, направленного и рассеянного света и материала выбранного объекта.

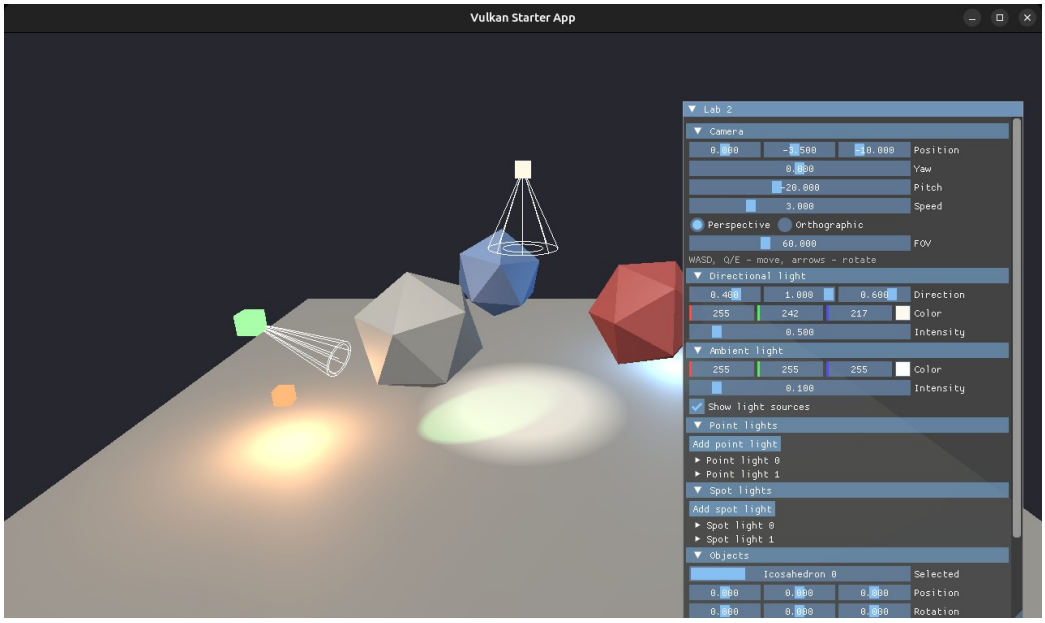


Рисунок 2 — итоговая сцена: к направленному источнику добавлены два точечных источника (оранжевый и голубой кубы) и два прожектора (белый и зелёный кубы с конусами из белых линий). На полу видны пятна света с затуханием по закону обратных квадратов, на икосаэдрах — блики.

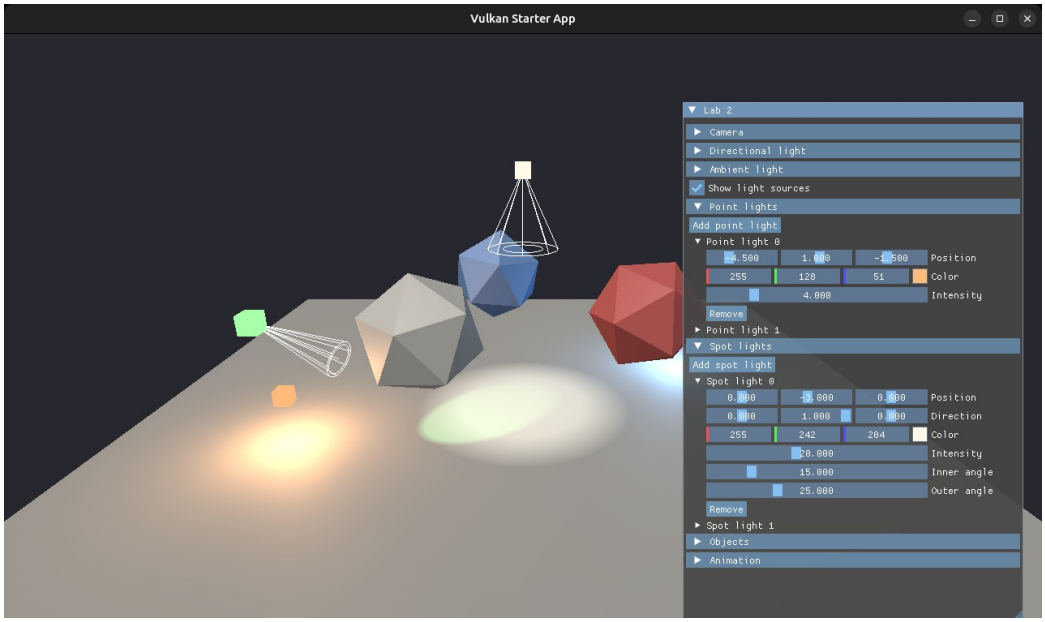


Рисунок 3 — параметры источников в интерфейсе: у точечного источника — позиция, цвет и интенсивность, у прожектора — ещё направление, внутренний и внешний углы; кнопки добавления и удаления источников и переключатель «Show light sources».

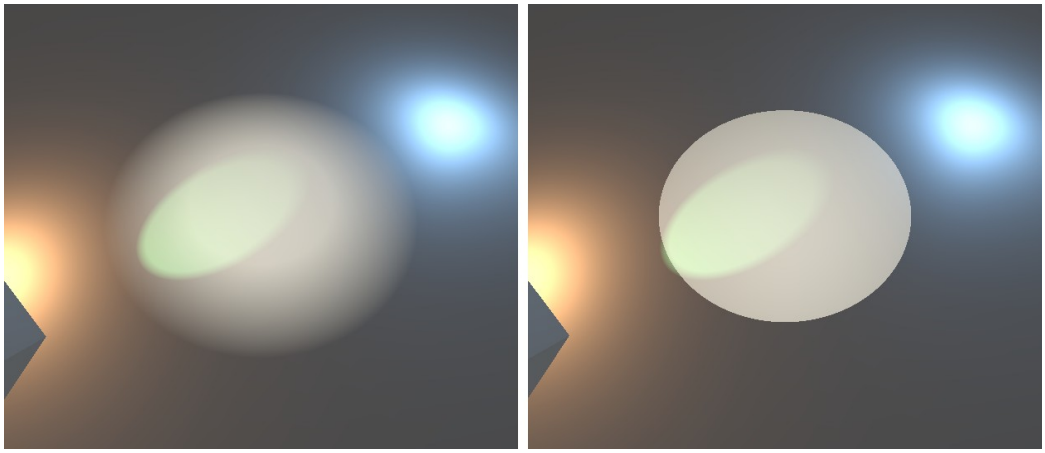


Рисунок 4 — пятно прожектора сверху (направленный источник выключен): слева внутренний угол 15°, внешний 30° — край мягкий; справа оба угла 25° — край резкий.

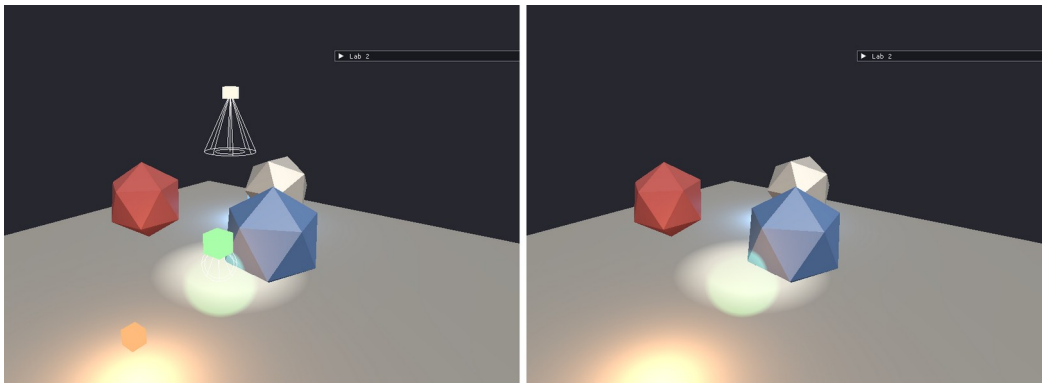


Рисунок 5 — вид сбоку: слева визуализация источников включена — кубы цвета источников и конусы прожекторов не освещаются и имеют постоянный цвет; справа визуализация скрыта.

Выводы

Я научился реализовывать камеру, которую можно перемещать и поворачивать, и строить для неё матрицу вида как обратную к матрице камеры. Разобрался, что такое нормали, как их получить процедурно через векторное произведение и как правильно переносить их в мировые координаты. Реализовал модель освещения Блинна-Фонга во фрагментном шейдере для направленного, точечных и прожекторных источников, затухание по закону обратных квадратов и мягкие края прожектора. Научился разделять данные шейдеров на наборы дескрипторов по частоте изменения, передавать массивы переменной длины через storage-буферы и учитывать правила выравнивания std140 и std430; использовал push-константы и несколько графических конвейеров для визуализации источников.